

**SIMATS ENGINEERING**  
**MACHINE LEARNING (ITA0601)**

**STUDENT ACADEMICS**

**FACULTY: DR.R.KAVITHA**

**NAME:HARSHAVARDHAN**

**REG NO: 192324189**

**QUESTION 1**

You are given a large dataset of patient health records stored in a CSV file, containing missing values, inconsistent formats, and outliers. Using NumPy and Pandas, design a constraint-based data cleaning pipeline that ensures: (i) no missing values remain, (ii) all numerical values fall within medically valid ranges, and (iii) categorical fields are standardized. Explain how you will use indexing, filtering, and aggregation functions to enforce these constraints while preserving maximum data integrity.

**SOLUTION**

A suitable approach is to implement the following steps:

- Load the CSV using Pandas and inspect structure, data types, missing values, and summary statistics using `df.info()`, `df.isnull().sum()`, and `df.describe()`.
- Use Pandas indexing and Boolean filtering to identify missing, invalid, and outlier values in each numerical field.
- For missing numerical values, use an appropriate statistic such as the median; for categorical values, use the mode or a defined valid category. The choice should be made according to the field and its meaning.
- Define medically valid minimum and maximum ranges for each numerical field. Filter values outside those ranges and replace, correct, or flag them according to the data-cleaning policy rather than silently deleting valid records.
- Standardize categorical fields by stripping extra spaces, converting values to a common case, and mapping different spellings or abbreviations to one standard category.
- Use NumPy operations such as `np.where()` for efficient conditional replacement or flagging of invalid values.
- Use aggregation functions such as `mean()`, `median()`, `min()`, `max()`, and `count()` after cleaning to verify that constraints are satisfied.
- Perform a final validation using `isnull().sum()`, range checks, and `unique()` values for categorical columns. Save the cleaned DataFrame only after all constraints pass.

## QUESTION 2

A system with limited memory must process a massive numerical dataset using NumPy arrays. You are required to perform mathematical operations, random sampling, and statistical analysis without exceeding memory limits. Propose a solution that uses efficient array creation, slicing, and in-place operations. Discuss how indexing and aggregation can be optimized to minimize memory usage, and explain debugging strategies if memory overflow or performance bottlenecks occur.

## SOLUTION

A suitable approach is to implement the following steps:

- Choose memory-efficient NumPy data types such as float32 or int32 when their precision and range are sufficient.
- Create arrays efficiently with functions such as `np.empty()`, `np.zeros()`, `np.ones()`, or `np.arange()`, avoiding repeated allocation of large temporary arrays.
- Process the massive dataset in chunks using slicing so that only the required portion is held in memory at a time.
- Use vectorized mathematical operations instead of Python loops. Where safe, use in-place operators such as `+=`, `-=`, `*=`, and `/=` to reduce temporary copies.
- Use NumPy random sampling to obtain a representative sample when full-data sampling is unnecessary, while keeping the sample size within the memory budget.
- Use aggregation functions such as `np.sum()`, `np.mean()`, `np.min()`, `np.max()`, and `np.var()` on chunks and combine results appropriately.
- Use direct indexing and slicing to access only required rows or columns instead of creating unnecessary copies.
- For debugging, inspect array shape, dtype, nbytes, and memory usage. Profile execution time, identify repeated allocations, and reduce chunk size if memory overflow occurs. Check vectorized expressions for unnecessary temporary arrays when performance is poor.

## QUESTION 2

Two large Pandas DataFrames containing customer transaction and demographic data must be combined. However, constraints include: (i) no duplication of customer IDs, (ii) consistent indexing after merging, and (iii) preservation of all valid records. Describe how you will perform the combining operation, validate constraints using filtering and grouping, and ensure correctness through sorting and indexing techniques. Include steps to debug mismatched or missing entries.

## SOLUTION

A suitable approach is to implement the following steps:

- Inspect both DataFrames and check customer IDs for missing values and duplicates using `isnull()`, `isna()`, and `duplicated()`.
- Standardize the customer ID column in both DataFrames so that data types and formatting are consistent before the merge.
- Use Pandas `merge()` with the join type appropriate to the requirement. An outer merge is suitable when valid records from both sources must be preserved.
- After merging, use `groupby('customer_id').size()` or equivalent aggregation to detect customer IDs that appear more times than expected.
- Use Boolean filtering to locate unmatched records, missing demographic information, or transactions without a corresponding customer record.
- Use `reset_index()` to establish a clean sequential index when appropriate, or `set_index()` when customer ID should be the index. Sort the final DataFrame using `sort_values()` or `sort_index()`.
- Compare row counts and unique customer-ID counts before and after merging to ensure valid records were not lost or duplicated.
- Debug mismatches by checking ID data types, spaces, capitalization, formatting, missing keys, duplicate keys, and join type. Inspect unmatched records separately before deciding how they should be handled.

## QUESTION 2

You are tasked with computing statistical measures (mean, median, variance) on a dataset using NumPy and Pandas, but with constraints on both computational efficiency and numerical accuracy. Explain how you will implement aggregation and statistical functions while avoiding precision loss and redundant computations. Discuss how filtering and indexing can be used to exclude irrelevant data points, and how you would debug incorrect statistical outputs.

## SOLUTION

A suitable approach is to implement the following steps:

- Load the data into Pandas and inspect data types, missing values, and invalid observations before computing statistics.
- Use Pandas aggregation functions such as `mean()`, `median()`, and `var()`, or NumPy functions `np.mean()`, `np.median()`, and `np.var()` on the required numerical columns.
- Use Boolean filtering and indexing to exclude irrelevant, invalid, or out-of-scope observations before aggregation.
- Avoid redundant calculations by cleaning and filtering the dataset once and reusing the resulting DataFrame or NumPy array.
- Use an appropriate numerical dtype and sufficient precision for the application. Avoid unnecessary conversions that may reduce numerical accuracy.
- Prefer vectorized NumPy/Pandas operations over explicit Python loops to improve computational efficiency.
- Check the variance definition carefully. Pandas and NumPy may use different default degrees of freedom, so specify the required parameter when comparing results.
- Debug incorrect results by checking the filtered rows, number of observations, data types, missing values, and a small manually calculated sample. Compare Pandas and NumPy outputs under the same settings.

## QUESTION 2

Design a complete data analysis pipeline that reads multiple data files, processes them using Pandas DataFrames, and generates visualizations. Constraints include: (i) efficient file I/O operations, (ii) correct grouping and sorting of data, (iii) filtering based on user-defined conditions, and (iv) meaningful plotting outputs. Explain how each stage (loading, processing, aggregation, and plotting) will be implemented and optimized, along with debugging steps for handling corrupted files or inconsistent data formats.

## SOLUTION

A suitable approach is to implement the following steps:

- Identify the input files and load them using appropriate Pandas functions such as `pd.read_csv()`. For very large files, use options such as `chunksize` to limit memory usage.
- Inspect each DataFrame for column names, data types, missing values, and inconsistent formats. Standardize column names and formats before combining files.
- Use `concat()` when files have compatible columns and represent additional rows; use `merge()` when related datasets must be combined using a common key.
- Apply user-defined conditions with Boolean filtering so that only relevant records are included in the analysis.
- Use `groupby()` with aggregation functions such as `sum()`, `mean()`, `count()`, and `median()` to create meaningful summaries.
- Use `sort_values()` or `sort_index()` to organize results before visualization and ensure the intended ordering is preserved.
- Generate meaningful plots such as bar charts, line charts, histograms, or scatter plots based on the analysis objective. Include clear titles, axis labels, and appropriate grouping.
- For debugging, catch file-reading errors, inspect corrupted files separately, compare schemas, verify date and numeric formats, check row counts after each stage, and inspect a sample of the final DataFrame before plotting.